

WORLD-CLASS MASTERCLASS NOTES

How the Internet & the Web Actually Work

The Unbreakable Foundation for Every Web Developer

Prepared for: Get Techy With Lucky / Tech Pulse Insider

Instructor: Lucky Nakola

WHAT YOU WILL MASTER IN THIS MASTERCLASS

MODULE 1 — The Global Plumbing: How physical cables, signals & packets carry information worldwide

MODULE 2 — The Digital Address Book: IP addresses, routers, DNS & how the web finds anything

MODULE 3 — The Conversation: How browsers & servers talk using HTTP/HTTPS requests & responses

MODULE 4 — The Canvas: How browsers transform HTML, CSS & JavaScript into living interfaces

By the end of this session, you will think like an engineer — not just a user.

You will understand the system you are building ON, not just building WITH.

Table of Contents

Table of Contents.....	2
Section 1 — The Core Masterclass Blueprint.....	4
The Four-Module Learning Arc.....	4
Part A — The No-Code Hook: The World's Most Expensive Post Office.....	5
Part B — Deep Technical Deconstruction.....	5
1. What Is the Internet?	6
2. The Physical Layer — What the Internet Is Actually Made Of	6
3. ISPs — Your On-Ramp to the Internet.....	6
4. Packets — The Core Innovation	7
5. What Is Inside a Packet?	7
6. Packet Switching vs Circuit Switching.....	8
7. The TCP/IP Protocol Suite — The Rules of the Road.....	8
Part C — The Go-Mode Activity: Human Packet Routing	8
Part A — The No-Code Hook: The World's Largest Phone Book	10
Part B — Deep Technical Deconstruction.....	10
1. IP Addresses — The Internet's Numbering System.....	10
2. Routers — The Intelligent Traffic Directors	11
3. DNS — The Domain Name System in Full Detail.....	11
4. Domain Names — The Human Layer	13
Part C — The Go-Mode Activity: DNS Detective	14
Part A — The No-Code Hook: The Restaurant	15
Part B — Deep Technical Deconstruction.....	15
1. The Client-Server Model	16
2. HTTP — HyperText Transfer Protocol	16
3. Anatomy of an HTTP Request.....	16
4. HTTP Methods — The Actions.....	17
5. Anatomy of an HTTP Response	17
6. HTTP Status Codes — The Server's Vocabulary	18
7. HTTPS — Adding the Security Layer.....	18
Part C — The Go-Mode Activity: The HTTP Conversation Roleplay	19
Part A — The No-Code Hook: The Architect, Decorator & Electrician.....	21
Part B — Deep Technical Deconstruction.....	21

1. What Is a Browser?	22
2. The Critical Render Path — Step by Step	22
3. JavaScript's Role — The Dynamic Layer.....	24
4. The Complete Journey — Wiring All Four Modules Together.....	24
Part C — The Go-Mode Activity: Build the Render Tree	25
Section 3 — The Ultimate Terminology Vault.....	27
Category 1 — Infrastructure	27
Category 2 — Addressing & Resolution	27
Category 3 — Protocols & Web Architecture	28
Section 4 — Interactive Assessment Engine	30
Part A — Scenario-Based Multiple Choice Questions	30
Part B — Open-Ended Systems Thinking Prompts	35
Instructor's Final Word	39

Section 1 — The Core Masterclass Blueprint

Before we touch a single line of code, we need to do something most beginner courses skip entirely: we need to understand the WORLD our code will live in. The internet is not magic. It is not a cloud floating in the sky. It is a physical, logical, meticulously engineered system — and the best engineers understand their environment deeply.

This masterclass is designed around one idea: engineer-level empathy. You will not just know what the internet is. You will feel how it works. You will trace a message from your fingertips to a server in another continent and back — and you will never look at a browser the same way again.

INSTRUCTOR NOTE — How to Use These Notes

These notes are designed for active delivery — not passive reading.

Every module has three layers:

1. The No-Code Hook — A real-world analogy to create an 'aha!' moment before any jargon.
2. Deep Technical Deconstruction — The actual system, explained with precision and clarity.
3. The Go-Mode Activity — A physical or logical simulation to lock in understanding.

Deliver the hook FIRST. Let students feel it before you name it.

Time per module: approximately 45–60 minutes (hook: 5 min, deconstruction: 25 min, activity: 20 min).

The Four-Module Learning Arc

Module	Title	Core Concepts
Module 1	The Global Plumbing	Physical internet, cables, ISPs, packets, packet switching
Module 2	The Digital Address Book	IP addresses, routers, DNS, domain names, registrars
Module 3	The Conversation	Client-server model, HTTP/HTTPS, requests, responses, status codes
Module 4	The Canvas	Browser rendering, DOM, CSSOM, JavaScript engine, critical render path

MODULE 1

The Global Plumbing

The Physical Internet, Cables, ISPs & Packets

Part A — The No-Code Hook: The World's Most Expensive Post Office

Here is the analogy that cracks the internet open for any beginner. Tell this story before touching any terminology.

INSTRUCTOR SCRIPT — Deliver This Out Loud

Imagine you write a letter to a friend who lives in Tokyo. You live in Nyeri, Kenya.

You cannot just throw the letter — it has to travel. So here is what actually happens:

1. You put your letter in an envelope and write the destination address on it.
2. Your local post office picks it up.
3. But your letter is too long to send as one piece — the post office CUTS it into 10 smaller envelopes, each with the same destination address and a number: Piece 1 of 10, Piece 2 of 10...
4. Each piece travels on a DIFFERENT route. Some go by truck. Some go by plane. Some even get rerouted if the highway is congested.
5. Every sorting office (hub) along the way reads the address and decides: next stop?
6. All 10 pieces arrive in Tokyo — probably in different order.
7. Your friend's local post office REASSEMBLES all 10 pieces into your original letter.
8. Your friend reads it.

That is EXACTLY how data travels across the internet.

Your letter = a data file. The pieces = packets. The post offices = routers.

The routes = network paths. Your friend = a web server.

Part B — Deep Technical Deconstruction

Now let us name the real things behind the analogy. Every term here maps directly to something in the story above.

1. What Is the Internet?

The internet is a global network of networks. It is not one thing owned by one company. It is millions of devices — computers, phones, servers, routers, switches — all connected by physical cables, fibre-optic lines, wireless signals, and satellite links. These devices agree to communicate using the same set of rules called protocols.

The internet's defining characteristic is that it is **DISTRIBUTED**. There is no single point of failure. If one route breaks, data finds another path. This was by design — the original ARPANET was built for the US military to survive a nuclear strike.

2. The Physical Layer — What the Internet Is Actually Made Of

Physical Medium	What It Is & How It Works
Copper Cables (Ethernet)	The original wiring. Still used in homes and offices. Carries electrical signals. Limited distance and speed compared to fibre.
Fibre-Optic Cables	The internet's superhighway. Carries data as pulses of LIGHT through glass threads thinner than a human hair. Speeds of 100+ Gbps. Submarine cables cross every ocean.
Submarine Cables	Thousands of kilometres of fibre-optic cable laid on the ocean floor. Over 400 of them connect every continent. They carry 95%+ of all intercontinental internet traffic.
Wireless (Wi-Fi / 4G / 5G)	The 'last mile' of the internet. Your phone connects wirelessly to a cell tower or router — but that tower connects to fibre-optic cables. Wireless gets you to the grid; fibre carries you the rest of the way.
Satellites (e.g. Starlink)	Low-Earth orbit satellites provide internet where cables cannot reach. Higher latency than fibre (signal travels to space and back) but revolutionary for remote areas in Africa.

3. ISPs — Your On-Ramp to the Internet

An ISP (Internet Service Provider) is the company that gives you access to the internet. Safaricom, Zuku, Airtel — these are ISPs. They own the physical infrastructure (cables, towers, data centres) that connects your home or phone to the global internet.

ISPs have a hierarchy: your local ISP connects to a national ISP, which connects to a global 'Tier 1' ISP (like Level 3, AT&T). Think of it like roads: your street connects to a district road, which connects to a national highway.

4. Packets — The Core Innovation

The single most important concept in networking. Every piece of data — a webpage, an email, a video, a WhatsApp message — is broken into small chunks called PACKETS before being sent across the network.

Why? Because sending one giant file on one dedicated path is inefficient. Breaking it into packets means:

- Each packet can travel on the best available route at that moment.
- The network can be shared by millions of users simultaneously.
- If one packet is lost, only that packet is resent — not the whole file.
- The destination reassembles packets in the correct order using sequence numbers.

5. What Is Inside a Packet?

NETWORK PACKET	
HEADER	PAYLOAD (Data)
Source IP: 41.90.64.1	[Actual content: part of a web page, image, video, email, etc.]
Dest IP: 142.250.74.78	
Packet #: 0047 of 0312	
Protocol: TCP	
Checksum: A3F7	

Source IP = where the packet came from

Dest IP = where it is going

Checksum = a fingerprint to detect corruption during transit

6. Packet Switching vs Circuit Switching

Method	How It Works & Pros/Cons
Circuit Switching	Traditional phone calls. A dedicated physical path is reserved for the entire duration of the call. Like reserving a road just for your car. Inefficient for data — the path sits idle when no data is flowing.
Packet Switching	The internet's method. No dedicated path. Each packet is routed independently through whatever path is available at that moment. Extremely efficient — the network is shared by everyone simultaneously.

7. The TCP/IP Protocol Suite — The Rules of the Road

For all these packets to work reliably, every device must follow the same rules. These rules are defined in the TCP/IP protocol suite — the foundation of all internet communication.

Protocol	Job & Function
IP (Internet Protocol)	Responsible for ADDRESSING and ROUTING. Gives every device a unique address (IP address) and determines the path packets take. Operates at the Network Layer.
TCP (Transmission Control Protocol)	Responsible for RELIABILITY. Breaks data into packets, numbers them, sends them, and ensures they all arrive and are reassembled correctly. If a packet is lost, TCP requests it again.
UDP (User Datagram Protocol)	Faster but less reliable than TCP. Used for streaming, gaming, video calls — where speed matters more than perfect delivery (a dropped frame in a video call is fine).

Part C — The Go-Mode Activity: Human Packet Routing

CLASSROOM SIMULATION — 20 Minutes — Energy Level: HIGH

SETUP:

- Designate 5 students as ROUTERS — each stands at a different point in the room.
- The instructor is the SOURCE (client).
- One student at the back is the DESTINATION (server).
- Print or write 5 index cards. Each card = one packet:

Card 1: [Source: Lucky | Dest: Server | Packet 3 of 5 | Data: 'ello W']

Card 2: [Source: Lucky | Dest: Server | Packet 1 of 5 | Data: 'H']

Card 3: [Source: Lucky | Dest: Server | Packet 5 of 5 | Data: '!']

Card 4: [Source: Lucky | Dest: Server | Packet 4 of 5 | Data: 'orld']

Card 5: [Source: Lucky | Dest: Server | Packet 2 of 5 | Data: 'i'] ← deliberately wrong

RULES:

- Each ROUTER receives a packet and must pass it to the next router toward the destination.
- ROUTERS cannot read the payload — they can only see Source and Dest IP.
- If a router drops a card (simulate packet loss), the destination must REQUEST it again.
- The destination reassembles ALL packets in NUMERICAL ORDER.

DEBRIEF QUESTIONS:

- What happens if packets arrive out of order? (TCP reassembles using sequence numbers)
- What if Router 3 is removed? (Packets find another path — resilience!)
- What does the checksum on each card simulate? (Data integrity verification)
- Did the message make sense before all packets arrived? (No — TCP waits for all pieces)

MODULE 2

The Digital Address Book

IP Addresses, Routers, DNS & Domain Names

Part A — The No-Code Hook: The World's Largest Phone Book

INSTRUCTOR SCRIPT — Deliver This Out Loud

Before phone numbers were stored in contacts apps, how did people find each other?

A PHONE BOOK. A giant directory: Name on the left, Number on the right.

'Nakola, Lucky — 0712 345 678'

The internet has exactly the same problem. Computers do not understand names like 'google.com'.

They understand numbers — specifically, IP addresses like 142.250.74.78.

But humans cannot memorise 142.250.74.78. We remember 'google.com'.

So the internet built a phone book: the DNS — Domain Name System.

DNS translates human-friendly names into machine-friendly numbers.

Every time you type a website address, a phone book lookup happens invisibly in milliseconds.

And just like a real phone book has a hierarchy (local book → city directory → national register),

DNS has a hierarchy too: your device → your ISP → root servers → domain servers.

Part B — Deep Technical Deconstruction

1. IP Addresses — The Internet's Numbering System

Every device connected to the internet has an IP (Internet Protocol) address — a unique numerical label. Think of it as a house address, but for computers.

IP Type	Description
IPv4	The original format. 32-bit number written as four groups of 0–255 separated by dots. Example: 192.168.1.1 or 142.250.74.78. Only ~4.3 billion addresses possible — nearly exhausted.
IPv6	The modern format. 128-bit number written in hexadecimal groups. Example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334. 340 undecillion addresses — essentially unlimited.
Public IP	Your address on the global internet — assigned by your ISP. Everyone on the internet can see your public IP when you make a request.
Private IP	Your address on your local network (home Wi-Fi). Your router assigns these internally. Examples: 192.168.x.x, 10.0.x.x. Not visible on the internet.
Static IP	A permanent, fixed IP address that never changes. Used by servers that need a consistent address.
Dynamic IP	An IP address that changes each time you connect. Most home users have dynamic IPs assigned automatically via DHCP.

2. Routers — The Intelligent Traffic Directors

A router is a device that reads the destination IP address of each packet and decides which path to send it on — toward its destination. Your home Wi-Fi router connects your devices to your ISP. Massive backbone routers in data centres direct billions of packets per second across continents.

Routers build and maintain ROUTING TABLES — databases of known network paths. When a packet arrives, the router consults its table and forwards the packet on the optimal path. If a path is congested or broken, the router automatically uses an alternative route.

3. DNS — The Domain Name System in Full Detail

DNS is the distributed, hierarchical naming system that translates domain names into IP addresses. It is not one server — it is a global network of millions of servers working together.

The DNS Hierarchy

DNS Server Type	Role in the Lookup Chain
Root Servers	The top of the hierarchy. 13 sets of root servers worldwide (operated by organisations like ICANN, Verisign). They know the addresses of all Top-Level Domain (TLD) servers.
TLD Servers	Manage top-level domains: .com, .ke, .org, .net. The .ke TLD is managed by KENIC (Kenya Network Information Centre).

DNS Server Type	Role in the Lookup Chain
Authoritative Name Server	The final authority for a specific domain. When you register 'mysite.co.ke', you configure an authoritative name server that maps 'mysite.co.ke' to your server's IP.
Recursive Resolver	Usually run by your ISP or Google (8.8.8.8) or Cloudflare (1.1.1.1). This is the server your device asks first. It does the hunting on your behalf and caches results.
DNS Cache	A temporary memory of recent lookups. Your browser, operating system, and recursive resolver all cache DNS results to avoid repeated lookups (controlled by TTL — Time to Live).

The Full DNS Resolution Journey — Step by Step

You type: `www.gettechy.co.ke` and press Enter.

STEP 1 — Browser Cache Check

Your browser asks: 'Have I looked up `gettechy.co.ke` recently?'
If YES → use cached IP → SKIP to making the request.
If NO → continue.

STEP 2 — Operating System Cache

The OS checks its own DNS cache and the local 'hosts' file.
If found → return IP. If not → continue.

STEP 3 — Recursive Resolver (your ISP's or Google's DNS: 8.8.8.8)

Your device asks: 'Hey recursive resolver, what is the IP for `gettechy.co.ke`?'
Resolver checks its cache. If not cached → continue.

STEP 4 — Root Server Query

Resolver asks a root server: 'Who handles `.ke` domains?'
Root server replies: 'KENIC handles `.ke`. Here is their address.'

STEP 5 — TLD Server Query (`.ke`)

Resolver asks KENIC: 'Who handles `gettechy.co.ke`?'
KENIC replies: 'The authoritative name server for `gettechy.co.ke` is `ns1.host.co.ke`'

STEP 6 — Authoritative Name Server Query

Resolver asks `ns1.host.co.ke`: 'What is the IP for `www.gettechy.co.ke`?'
Authoritative server replies: '`196.201.214.50`' ← THE ACTUAL IP!

STEP 7 — IP Returned & Cached

Recursive resolver caches the result (for the duration of TTL).
Returns 196.201.214.50 to your browser.

STEP 8 — Browser Now Has the IP

Can finally send an HTTP request to 196.201.214.50.

Total time for all this: typically 20-120 milliseconds.

4. Domain Names — The Human Layer

A domain name is the human-readable address of a website. It has a structure that reads right to left by level of hierarchy:

```
www.gettechy.co.ke
|   |   |   |
|   |   |   └─ Country Code TLD (.ke = Kenya)
|   |   └─── Second-Level TLD (.co = commercial in Kenya)
|   └────── Second-Level Domain (gettechy — your unique name)
└────────── Subdomain (www — a subdivision of the domain)
```

Other examples:

mail.google.com → subdomain: mail, domain: google, TLD: .com

api.github.com → subdomain: api, domain: github, TLD: .com

portal.kra.go.ke → subdomain: portal, domain: kra, TLD: .go.ke
(Kenyan govt)

DNS Concept	What It Is
Domain Registrar	Company where you purchase and register a domain name. Examples: GoDaddy, Namecheap, Kenya Website Experts, Truehost Kenya.
DNS Records	Configuration entries that map your domain to services. The most important types:
A Record	Maps domain to an IPv4 address. gettechy.co.ke → 196.201.214.50
AAAA Record	Maps domain to an IPv6 address.
CNAME Record	Creates an alias. www.gettechy.co.ke → gettechy.co.ke
MX Record	Points to mail server — controls where email is delivered.
TXT Record	Text records used for verification (Google, SSL, spam prevention).
TTL (Time to Live)	How long DNS resolvers cache your record before checking again. Lower TTL = faster propagation of changes but more DNS queries.

Part C — The Go-Mode Activity: DNS Detective

CLASSROOM ACTIVITY — 20 Minutes — Roles, Cards & Whiteboards

SETUP:

- Assign roles: 1 Root Server, 2 TLD Servers (.com and .ke), 3 Authoritative Servers, 2 Recursive Resolvers.

- Each server writes their 'database' on a card:

Root Server card: '.com → go to TLD_COM | .ke → go to TLD_KE'

TLD_COM card: 'google.com → go to AUTH_GOOGLE | github.com → go to AUTH_GITHUB'

AUTH_GOOGLE card: 'www.google.com → 142.250.74.78'

THE QUERY GAME:

- A student (Browser) holds a paper saying: 'I need the IP for www.google.com'.
- They go to the Recursive Resolver. The resolver asks the Root Server. Root → TLD → Auth.
- The resolved IP is passed back to the Browser.
- Run 3 rounds. Second round: ask for a CACHED result (resolver remembers).
- Third round: the auth server has CHANGED the IP — the old cached result is now wrong.
Discuss: What is TTL? Why does DNS propagation take time?

CHALLENGE EXTENSION:

- Open a real terminal and type: `nslookup www.google.com`
- Or on the web: dnschecker.org — watch DNS propagation live across the globe.
- Question: Why might google.com resolve to a DIFFERENT IP in Kenya vs the USA?
Answer: Google has servers everywhere. DNS is used for load balancing and geo-routing!

MODULE 3

The Conversation

The Client-Server Model, HTTP/HTTPS, Requests & Responses

Part A — The No-Code Hook: The Restaurant

INSTRUCTOR SCRIPT — Deliver This Out Loud

You walk into a restaurant. Here is the complete cast of characters:

YOU = the CLIENT (your browser)

THE WAITER = HTTP (the protocol — the agreed language of communication)

THE KITCHEN = the SERVER (the computer that has the food/data)

THE MENU = the URL (tells you what is available and where)

YOUR ORDER = the HTTP REQUEST ('GET me the chips, please')

THE FOOD = the HTTP RESPONSE (the server sends you what you asked for)

Now watch how the conversation actually goes:

You: 'Excuse me — I would like the grilled chicken, table 7.'

Waiter writes it down (formats the request), takes it to the kitchen.

Kitchen checks: 'Do we have grilled chicken? YES.' Prepares it. Sends it out.

Waiter brings it: 'Here you go — grilled chicken for table 7. Status: 200 OK.'

What if the kitchen does NOT have what you ordered?

Waiter comes back: 'Sorry, we are OUT of that. Status: 404 — Item Not Found.'

What if the kitchen caught fire?

'Sorry, there is an internal emergency. Status: 500 — Server Error.'

Every single time you load a webpage, this exact conversation happens.

Part B — Deep Technical Deconstruction

1. The Client-Server Model

The internet is built on a fundamental division of roles: clients and servers. This is not just a metaphor — it is an architectural pattern that defines how almost every web interaction works.

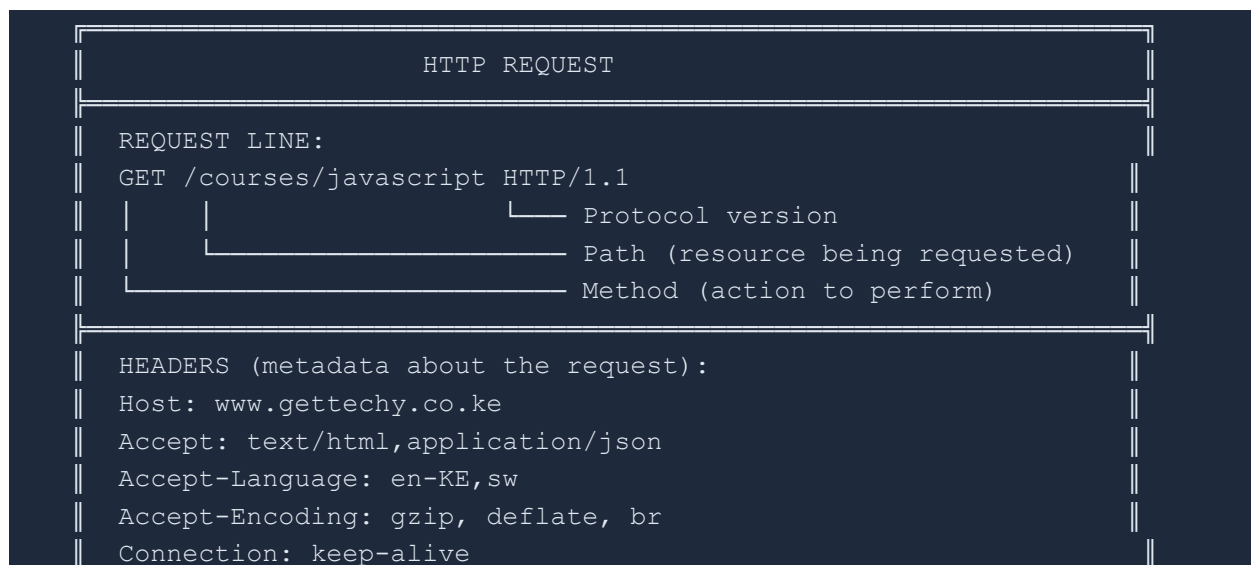
Role	What It Does
Client	Any device or software that REQUESTS information or services. Your browser (Chrome, Firefox, Safari) is a client. Your phone's apps are clients. A client initiates the conversation.
Server	A computer (often a powerful one in a data centre) that PROVIDES information or services in response to requests. Web servers, API servers, database servers — all servers wait for requests and respond.
The Key Rule	Clients initiate. Servers respond. A server never sends data unless a client asked first (except with modern WebSockets — but that comes later).

2. HTTP — HyperText Transfer Protocol

HTTP is the language — the protocol — that clients and servers use to communicate on the web. It is a set of rules that defines:

- How a client formats a request for data
- How a server formats a response
- What methods (actions) are available (GET, POST, PUT, DELETE, PATCH...)
- What headers (metadata) must accompany each message
- What status codes mean in the response

3. Anatomy of an HTTP Request




```
User-Agent: Mozilla/5.0 (Chrome/125...)
Cookie: session_id=abc123; theme=dark

BODY (only for POST, PUT, PATCH):
{ 'name': 'Lucky', 'email': 'lucky@example.com' }
```

4. HTTP Methods — The Actions

Method	What It Does	Real-World Example
GET	Retrieve data. No body. Safe & idempotent.	Load a webpage, fetch an API response
POST	Send new data to server. Has a body.	Login form, register form, create a post
PUT	Replace an entire resource with new data.	Update a full user profile
PATCH	Partially update a resource.	Change just your profile photo
DELETE	Remove a resource from the server.	Delete an account, remove a blog post
HEAD	Like GET but returns only headers, no body.	Check if a resource exists without downloading it
OPTIONS	Ask what methods are supported for a resource.	CORS preflight checks in browsers

5. Anatomy of an HTTP Response

```
HTTP RESPONSE

STATUS LINE:
HTTP/1.1 200 OK
  |           |
  |           |—— Human-readable status message
  |           |—— Status code (the most important part)
  |           |—— Protocol version

RESPONSE HEADERS:
Content-Type: text/html; charset=UTF-8
Content-Length: 48291
Set-Cookie: session_id=xyz789; HttpOnly; Secure
Cache-Control: max-age=3600
X-Powered-By: PHP/8.2
Strict-Transport-Security: max-age=31536000

RESPONSE BODY:
<!DOCTYPE html>
```

```
<html lang='en'>
  <head><title>Get Techy With Lucky</title></head>
  <body><h1>Welcome!</h1>...</body>
</html>
```

6. HTTP Status Codes — The Server's Vocabulary

Status codes are three-digit numbers that tell the client exactly what happened with their request. Every web developer must memorise these groups and the most important individual codes.

Code Group	What It Means & Key Examples	Key Codes
1xx — Informational	100 Continue — keep sending your request, it is fine so far.	100, 101
2xx — Success	200 OK — the most beautiful words in web development. Request succeeded, here is the data. 201 Created — your POST worked and a new resource was created. 204 No Content — success but nothing to return.	200, 201, 204
3xx — Redirection	301 Moved Permanently — this URL has changed forever, update your bookmarks. 302 Found — temporary redirect. 304 Not Modified — your cached version is still current, no need to re-download.	301, 302, 304
4xx — Client Error	400 Bad Request — your request was malformed. 401 Unauthorised — you need to log in. 403 Forbidden — you are logged in but not allowed. 404 Not Found — the resource does not exist. 429 Too Many Requests — slow down, you are being rate-limited.	400, 401, 403, 404, 422, 429
5xx — Server Error	500 Internal Server Error — something broke on the server side (the kitchen caught fire). 502 Bad Gateway — a proxy got a bad response from the upstream server. 503 Service Unavailable — the server is overwhelmed or under maintenance.	500, 502, 503

7. HTTPS — Adding the Security Layer

HTTP sends everything in plain text — anyone intercepting the packets can read your password, credit card number, or personal messages. This was a disaster waiting to happen. HTTPS (HTTP Secure) solves this by encrypting the entire conversation.

HTTPS Concept	Explanation
SSL/TLS	The encryption protocol underneath HTTPS. TLS (Transport Layer Security) is the modern version. It encrypts data so that even if packets are intercepted, they look like random noise without the decryption key.

HTTPS Concept	Explanation
SSL Certificate	A digital certificate issued by a Certificate Authority (CA) that proves the server is who it claims to be. When you see the padlock in your browser, a valid certificate is present.
Certificate Authority	Trusted third parties (Let's Encrypt, DigiCert, Comodo) that verify website ownership and issue SSL certificates. Your browser comes pre-loaded with a list of trusted CAs.
TLS Handshake	The process where client and server agree on encryption settings, verify identity, and establish a secure channel — before any HTTP data is sent. Happens in milliseconds.
Mixed Content	A security warning: the page is HTTPS but some resources (images, scripts) are loaded over HTTP. Browsers block these as they compromise the security of the page.

Part C — The Go-Mode Activity: The HTTP Conversation Roleplay

CLASSROOM ROLEPLAY — 20 Minutes — Full Conversation Simulation

ROLES: 1 Browser (client), 1 Web Server, 1 DNS Resolver, 1 HTTPS Security Guard.

SCENARIO: A student (Browser) wants to open <https://www.gettechy.co.ke/courses>

SCRIPT TO ACT OUT:

Browser → DNS: 'What is the IP for www.gettechy.co.ke?'

DNS → Browser: 'The IP is 196.201.214.50.'

Browser → Security Guard (TLS): 'I want to talk to [gettechy.co.ke](https://www.gettechy.co.ke) securely.'

Security Guard checks certificate → 'Certificate valid. Encryption established. Go ahead.'

Browser → Server: writes on paper:

GET /courses HTTP/1.1

Host: www.gettechy.co.ke

User-Agent: Chrome/125

Accept: text/html

Server reads the note, thinks for 2 seconds, writes back:

HTTP/1.1 200 OK

Content-Type: text/html

[Body: the HTML of the courses page]

VARIATIONS (run multiple rounds):

Round 2: Browser asks for /courses/secret — Server replies: 401 Unauthorised

Round 3: Browser asks for /courses/php — Server replies: 404 Not Found

Round 4: Server crashes (student sits down) — what status code? → 503 Service Unavailable

DEBRIEF: Why must the security handshake happen BEFORE the request? (Encrypt everything)

Why does the browser include a User-Agent header? (Server can tailor response for device type)

MODULE 4

The Canvas

How Browsers Transform HTML, CSS & JS into Living Interfaces

Part A — The No-Code Hook: The Architect, Decorator & Electrician

INSTRUCTOR SCRIPT — Deliver This Out Loud

You are building a house from scratch. You have three specialists:

THE ARCHITECT — designs the structure. Decides where the rooms go, where the walls stand,

how many windows, where the doors are. No aesthetics — just raw structure.

In the browser: this is HTML.

THE INTERIOR DESIGNER — takes the raw structure and makes it beautiful. Chooses paint colours, carpet types, furniture styles. Cannot add rooms — works WITH the structure.

In the browser: this is CSS.

THE ELECTRICIAN — makes things MOVE and FUNCTION. Switches that turn lights on, doors

that open automatically, intercoms that respond to voice. Cannot design — adds behaviour.

In the browser: this is JavaScript.

Here is the crucial insight:

A house with no architect (no HTML) → nowhere to put anything.

A house with no designer (no CSS) → functional but ugly.

A house with no electrician (no JS) → beautiful but static.

The browser's job is to take all three and RENDER them into something you can see and use.

It does this through a process called the Critical Render Path — and it is fascinating.

Part B — Deep Technical Deconstruction

1. What Is a Browser?

A browser (Chrome, Firefox, Safari, Edge) is a sophisticated software application that does three things: fetches resources from the web, interprets code (HTML, CSS, JavaScript), and renders a visual interface you can interact with. At its core, a browser is a code interpreter and a rendering engine.

Browser Component	What It Does
Rendering Engine	Interprets HTML and CSS, builds the layout, and paints pixels on screen. Chrome/Edge use Blink, Firefox uses Gecko, Safari uses WebKit.
JavaScript Engine	Interprets and executes JavaScript code. Chrome's V8 engine is so fast it is also used in Node.js (server-side JS). Firefox uses SpiderMonkey.
Network Layer	Handles all HTTP/HTTPS requests — fetching HTML, CSS, JS, images, fonts, and API data.
UI Backend	The code that actually draws elements on screen using the operating system's graphics APIs.
Storage	Manages cookies, localStorage, sessionStorage, IndexedDB, and cache.

2. The Critical Render Path — Step by Step

The Critical Render Path (CRP) is the sequence of steps the browser must complete before you see ANYTHING on screen. Understanding this is why top engineers care so deeply about performance.

THE CRITICAL RENDER PATH

STEP 1 — Parse HTML → Build the DOM

Browser reads HTML top-to-bottom, byte by byte.

Converts tags into NODES. Assembles them into a tree: the DOM.

DOM = Document Object Model = the browser's in-memory map of the page.

```
<html>
  <body>
    <h1>Hello</h1>
    <p>World</p>
  </body>
```

```
</html>
```

DOM Tree:

Document

└─ html

└─ body

└─ h1 ('Hello')

└─ p ('World')

STEP 2 — Parse CSS → Build the CSSOM

Browser reads all CSS (from <link> tags, <style> blocks, inline styles).

Builds the CSSOM (CSS Object Model) — a similar tree structure.

CSS is RENDER-BLOCKING: the browser will not show anything until all CSS

is parsed. This is why large CSS files slow down page load.

STEP 3 — Combine DOM + CSSOM → Build the Render Tree

The browser merges the DOM and CSSOM.

Only VISIBLE elements are included (display:none elements are excluded).

Every node in the Render Tree knows its visual properties.

STEP 4 — Layout (Reflow)

The browser calculates the EXACT position and size of every element.

'The h1 starts at x=16, y=32, width=740px, height=48px.'

This uses the box model: content + padding + border + margin.

Layout is expensive — avoid triggering it repeatedly with JavaScript.

STEP 5 — Paint

The browser converts layout information into actual pixels.

Text, colours, borders, shadows, images — all converted to pixel data.

Happens on multiple 'layers' (like Photoshop layers).

STEP 6 — Composite

The layers are assembled in the correct order and displayed on screen.

Hardware-accelerated: your GPU does this step — that is why transforms

and opacity animations are smooth (GPU-handled, not CPU-handled).

3. JavaScript's Role — The Dynamic Layer

JavaScript is parsed and executed by the browser's JS engine. Crucially, JavaScript is **PARSER-BLOCKING** by default: when the browser encounters a `<script>` tag while parsing HTML, it **STOPS**, downloads the script, executes it, then continues. This is why we put scripts at the end of `<body>` or use 'defer'.

Script Type / Concept	Behaviour & When to Use
Normal <code><script></code>	STOPS HTML parsing. Downloads and executes immediately. Blocks everything. Avoid unless the script absolutely must run first.
<code><script defer></code>	Downloads in parallel. Executes AFTER HTML is fully parsed. Does not block. Maintains script order. Best practice for most scripts.
<code><script async></code>	Downloads in parallel. Executes AS SOON as download completes — can interrupt HTML parsing. Best for truly independent scripts (analytics).
JavaScript & Layout	JS can read layout info (<code>offsetWidth</code> , <code>getBoundingClientRect</code>). Reading and then writing layout in a loop causes 'layout thrashing' — a major performance problem.

4. The Complete Journey — Wiring All Four Modules Together

A STUDENT IN NYERI TYPES: <https://www.gettechy.co.ke/javascript>

[MODULE 1 — THE PHYSICAL INTERNET]

Request is broken into TCP/IP packets. Travels through Safaricom's fibre network, across submarine cables if needed, to the server's data centre. Packets find their own routes. Reassembled at destination.

[MODULE 2 — DNS]

Before packets can travel, browser asks DNS: 'IP of `gettechy.co.ke`?'
Root server → .ke TLD server → Authoritative server → IP returned.
Entire process: ~50ms. Result cached for TTL duration.

[MODULE 3 — HTTP/HTTPS]

TLS handshake establishes encrypted connection. (~100ms)
Browser sends: `GET /javascript HTTP/1.1` [with headers]
Server processes request, fetches data, sends back:
`HTTP/1.1 200 OK` [with headers + HTML body]

[MODULE 4 — BROWSER RENDERING]

HTML received → DOM built.
CSS received (may block) → CSSOM built.
JavaScript executed → may modify DOM or CSSOM.

DOM + CSSOM → Render Tree → Layout → Paint → Composite.
STUDENT SEES THE PAGE. (~Total: 200–800ms from keypress to pixels)

Part C — The Go-Mode Activity: Build the Render Tree

CLASSROOM ACTIVITY — 20 Minutes — Physical DOM/CSSOM Construction

MATERIALS NEEDED: index cards, coloured markers, string or tape.

STEP 1 — HTML Cards:

Write each HTML element on a separate card: html, head, body, header, nav, main, h1, p, ul, li, footer.

Connect them with string to form the DOM TREE structure on the floor or whiteboard.

STEP 2 — CSS Cards:

Write CSS rules on separate coloured cards:

h1: { color: navy; font-size: 48px; }

p: { font-size: 18px; line-height: 1.6; }

li: { display: none; } ← The hidden one!

STEP 3 — Merge into Render Tree:

Apply each CSS card to its matching DOM card.

The 'li' card gets a RED X — it is NOT included in the Render Tree (display: none).

Remaining cards form the RENDER TREE.

STEP 4 — Layout Round:

Each student who holds a Render Tree card must call out their element's approximate position on an imaginary screen: 'h1: top-left, full width, 48px tall'.

STEP 5 — JavaScript Disruption:

Instructor says: 'JavaScript just ran document.querySelector("h1").style.color = "red"'

The student holding the h1 card must update their CSS card — and explain:

'The browser now has to REPAINT just this element. That is why DOM manipulation can be expensive.'

FINAL DEBRIEF QUESTION:

What happens if CSS is still downloading when JavaScript tries to read an element's width?

Answer: JS execution is **BLOCKED** until CSSOM is complete — CSS blocks JS which blocks HTML.

This is the critical dependency chain that every performance engineer wrestles with.

Section 3 — The Ultimate Terminology Vault

Every term below is a weapon in your engineering vocabulary. Do not memorise them — understand them. Learn the WHY behind each one and the rest will follow.

Category 1 — Infrastructure

Term	Punchy Definition
Internet	A global network of networks. Not one company, not one cable — a distributed, self-healing system of billions of connected devices.
ISP	Internet Service Provider. The company that connects you to the internet. Safaricom, Zuku, Airtel are Kenyan ISPs.
Packet	A small chunk of data with a header (addressing info) and payload (actual content). All internet data travels as packets.
Packet Switching	The method of routing packets independently across the best available path — rather than reserving a dedicated circuit.
TCP/IP	The core protocol suite of the internet. TCP handles reliability (reassembly, error checking). IP handles addressing and routing.
Router	A device that reads packet destination IPs and forwards them toward their destination. The traffic director of the internet.
Fibre-Optic Cable	Glass threads that carry data as pulses of light. Faster and higher capacity than copper. The backbone of the global internet.
Submarine Cable	Undersea fibre-optic cables connecting continents. Carry 95%+ of intercontinental internet traffic.
Latency	The time delay between sending a request and receiving a response. Measured in milliseconds. The enemy of real-time applications.
Bandwidth	The maximum amount of data that can travel through a connection per second. NOT speed — speed is latency. Think motorway lanes vs speed limit.
Protocol	An agreed-upon set of rules for communication. HTTP, HTTPS, TCP, UDP, FTP, SMTP are all protocols.

Category 2 — Addressing & Resolution

Term	Punchy Definition
IP Address	The unique numerical identifier of a device on a network. IPv4 (e.g. 192.168.1.1) or IPv6. The machine-level address.
DNS	Domain Name System. The internet's phonebook. Translates human-readable domain names into IP addresses.
Domain Name	The human-friendly address of a website. Structured hierarchically: subdomain.domain.TLD (e.g. www.gettechy.co.ke).
TLD	Top-Level Domain. The rightmost part of a domain name. .com, .ke, .org, .net. The TLD tells you the domain's category or country.
DNS Record	A configuration entry that maps a domain to an IP or service. Types: A, AAAA, CNAME, MX, TXT, NS.
TTL	Time to Live. How long DNS resolvers cache a record before fetching a fresh copy. Low TTL = fast propagation; high TTL = fewer queries.
Recursive Resolver	The DNS server your device asks first. It hunts through the DNS hierarchy on your behalf and caches results.
Authoritative Server	The DNS server with the final, definitive answer for a domain. Set up by the domain owner.
CDN	Content Delivery Network. A global network of servers that cache copies of your website near users, reducing latency dramatically.
Hosting	Renting server space where your website files live. The server delivers your files to browsers on demand.
URL	Uniform Resource Locator. The complete address of a web resource: protocol + domain + path + query string.

Category 3 — Protocols & Web Architecture

Term	Punchy Definition
HTTP	HyperText Transfer Protocol. The language of the web. Defines how requests and responses are structured between clients and servers.
HTTPS	HTTP Secure. HTTP with TLS encryption. Protects data in transit. Required for any site handling personal or payment data.
TLS/SSL	Transport Layer Security (successor to SSL). The encryption protocol behind HTTPS. Creates an encrypted tunnel between client and server.
SSL Certificate	Digital proof of a server's identity, issued by a Certificate Authority. Enables HTTPS. Let's Encrypt provides free certificates.
HTTP Method	The ACTION of an HTTP request. GET (retrieve), POST (create), PUT (replace), PATCH (update), DELETE (remove).

Term	Punchy Definition
Status Code	A three-digit number in an HTTP response indicating the outcome. 200 = OK, 404 = Not Found, 500 = Server Error.
Request Header	Metadata sent with an HTTP request: Host, User-Agent, Accept, Authorization, Cookie, Content-Type.
Response Header	Metadata sent with an HTTP response: Content-Type, Cache-Control, Set-Cookie, Content-Length.
Client	Any device/software that initiates HTTP requests. Your browser, a mobile app, or a cURL command are all clients.
Server	Computer that receives requests, processes them, and returns responses. Runs software like Apache, Nginx, or Node.js.
API	Application Programming Interface. A set of rules and endpoints allowing software to communicate. REST APIs use HTTP.
JSON	JavaScript Object Notation. A lightweight text format for transmitting structured data between client and server.
Cookie	A small piece of data the server sends to the browser to store. Sent back with every subsequent request to the same domain.
Session	Server-side storage of user state (who is logged in). Identified by a session ID sent to the browser as a cookie.
Cache	Stored copies of resources (HTML, CSS, images) to reduce repeated network requests and improve load speed.
DOM	Document Object Model. The browser's in-memory tree representation of an HTML page. JavaScript manipulates it to change pages dynamically.
CSSOM	CSS Object Model. The browser's internal representation of all CSS rules. Merged with DOM to create the Render Tree.
Critical Render Path	The sequence of steps the browser must complete (parse HTML, parse CSS, execute JS, layout, paint, composite) before displaying a page.
Rendering Engine	The browser component that converts HTML+CSS into pixels. Blink (Chrome/Edge), Gecko (Firefox), WebKit (Safari).
JavaScript Engine	The browser component that executes JavaScript. V8 (Chrome/Node.js), SpiderMonkey (Firefox).

Section 4 — Interactive Assessment Engine

These assessments are diagnostic — designed to reveal depth of understanding, not test memorisation. Encourage students to reason aloud. Wrong answers are as instructive as correct ones.

Part A — Scenario-Based Multiple Choice Questions

Instructions: Select the BEST answer. Read each scenario carefully — the devil is in the details.

QUESTION 1 — Network Routing

Amira is in Mombasa, streaming a YouTube video from a server in Los Angeles. Halfway through the video, the undersea fibre-optic cable route through South Africa suffers a cut. What is the most accurate description of what happens?

- A) The video immediately freezes and cannot continue until the cable is repaired.
- B) The video buffers briefly as YouTube's CDN activates a backup copy closer to Amira.
- C) YouTube detects the broken route and automatically re-routes the video packets through a different path (via Asia-Pacific submarine cables) with minimal interruption.
- D) The video continues without issue because Amira already downloaded the whole file.

ANSWER KEY — Question 1

CORRECT ANSWER: C

WHY C IS CORRECT:

Packet-switched networks are inherently resilient. Routers continuously update their routing tables.

When a cable fails, routers detect the broken path and automatically redirect packets through alternative routes — often within seconds. The internet was designed SPECIFICALLY for this scenario.

WHY A IS WRONG:

This describes circuit switching (like old phone lines) — where one broken path = total failure. The internet uses packet switching precisely to avoid this single-point-of-failure problem.

WHY B IS PARTIALLY RIGHT but not BEST:

CDNs do help but the core mechanism described (CDN activation) is a content delivery optimisation, not the routing-layer answer. The question is about NETWORK routing.

WHY D IS WRONG:

Video streaming uses buffering, not full pre-download. And even if buffered, the reason for continuation would be the buffer, not the network — which is a different mechanism entirely.

QUESTION 2 — DNS Troubleshooting

A developer just updated their website's DNS A Record to point to a new server IP address. They immediately test the change from their laptop — but still see the OLD website. Their colleague in Nairobi CBD can already see the NEW website. What is happening?

- A) The developer made an error in the DNS record — it was not saved correctly.
- B) The developer's browser has cached the old HTML and needs to be hard-refreshed.
- C) DNS propagation has not completed — the developer's local DNS resolver is still serving a cached copy of the old IP address (TTL has not yet expired).
- D) The ISP is blocking access to the new server.

ANSWER KEY — Question 2

CORRECT ANSWER: C

WHY C IS CORRECT:

DNS resolvers cache records for the duration of the TTL (Time to Live). If the old record had a TTL of 3600 seconds (1 hour), every resolver that looked it up in the past hour is serving the cached old IP. DNS propagation is the process of all resolvers worldwide refreshing their caches. It can take minutes to 48 hours depending on TTL settings.

The developer's resolver has the old cached IP. The Nairobi colleague's resolver refreshed first.

WHY A IS WRONG:

If it were a record error, nobody would see the new site — not even the colleague in Nairobi.

WHY B IS POSSIBLE but NOT the BEST answer:

Browser caching is a real thing, but the scenario describes two different LOCATIONS seeing different results — which is only explainable by DNS caching, not browser caching.

WHY D IS UNLIKELY:

ISP blocking would affect all users, not selectively — and would not explain the colleague seeing it.

REAL-WORLD FIX: Use a low TTL (300 seconds) BEFORE making DNS changes. After propagation,

increase TTL again for performance. You can also test using a different DNS resolver:

`nslookup yourdomain.com 8.8.8.8` (forces use of Google's DNS resolver)

QUESTION 3 — HTTP Status Codes

A junior developer is building a school portal. A student logs in successfully, then tries to access `/admin/dashboard`. The server checks the student's role and denies access.

Which HTTP status code should the server return, and why?

- A) 404 Not Found — the admin dashboard should be hidden from students entirely.
- B) 401 Unauthorised — the student has not proven their identity.
- C) 403 Forbidden — the student IS authenticated but does NOT have permission for this resource.
- D) 302 Found — redirect the student back to their own dashboard.

ANSWER KEY — Question 3

CORRECT ANSWER: C

WHY C IS CORRECT:

403 Forbidden is the precise code for: 'I know who you are, but you are not allowed here.'

Authentication (who are you?) has passed — the student is logged in.

Authorisation (what can you do?) has FAILED — they do not have admin privileges.

403 = Authenticated but not Authorised. This is a critical security distinction.

WHY B IS WRONG:

401 Unauthorised means: 'You have not identified yourself. Please log in first.'

But the student IS logged in — they HAVE been authenticated. 401 would be incorrect here.

Using 401 could mislead the client into thinking their credentials are invalid.

WHY A IS A VALID SECURITY PRACTICE but wrong status code:

Returning 404 for protected resources is a security pattern (security through obscurity) that prevents attackers from knowing protected routes exist. However, 403 is the technically correct answer for this specific scenario — the question asks what 'should' be returned.

WHY D IS NOT A STATUS CODE ANSWER:

302 is a redirect, not an error. While you MIGHT redirect after a 403, D describes a 302 redirect, not the status code for the denial itself.

QUESTION 4 — Browser Rendering

A developer notices their webpage takes 4 seconds to show ANYTHING on screen, even though

the HTML itself is tiny. The page links to 5 large CSS files and 3 JavaScript files in the <head> section. What is the most likely cause, and what is the fix?

- A) The server is too slow — they need a more powerful server.
- B) CSS and JavaScript in the <head> are render-blocking — the browser will not paint anything until all CSS is parsed and all scripts are executed.
- C) The images on the page are too large — they are blocking the initial render.
- D) The browser has run out of memory and is struggling to process the page.

ANSWER KEY — Question 4

CORRECT ANSWER: B

WHY B IS CORRECT:

CSS is ALWAYS render-blocking: the browser cannot build the Render Tree without the CSSOM,

so it WILL NOT paint anything until all linked CSS files are downloaded and parsed.

JavaScript in <head> without 'defer' or 'async' is PARSER-BLOCKING: it stops HTML parsing,

downloads the script, executes it, then resumes. Five CSS files + three blocking JS files in the head = the browser is frozen for the entire download+parse time of all 8 files.

THE FIX:

1. Move `<script>` tags to the bottom of `<body>`, OR add 'defer' attribute.
2. Combine CSS files to reduce HTTP requests.
3. Use Critical CSS: inline the above-the-fold CSS in `<style>` tags, lazy-load the rest.
4. Use HTTP/2 to allow multiple files to download in parallel over one connection.

WHY C IS WRONG:

Images are NOT render-blocking. The browser can paint the page layout and show text while images still load in the background. Images do not block initial render.

WHY A IS UNLIKELY given the scenario:

The HTML is described as tiny — a slow server would manifest as high Time to First Byte, but the blocking described is happening CLIENT-SIDE after the HTML arrives.

QUESTION 5 — HTTPS & Security

A new developer builds a login page and deploys it over plain HTTP (not HTTPS).

They argue: 'It is fine — we are using a secure POST request, so the password is in the request body, not the URL. Nobody can see it.' How do you respond?

- A) They are correct — POST requests are always encrypted regardless of HTTP vs HTTPS.
- B) They are partially right — POST body is hidden in the URL but an attacker monitoring network traffic can read the body in plain text since HTTP has no encryption.
- C) They are wrong because GET requests should be used for login forms, not POST.
- D) They are correct — modern routers automatically encrypt all traffic.

ANSWER KEY — Question 5

CORRECT ANSWER: B

WHY B IS CORRECT — and this is a CRITICAL security understanding:

HTTP = NO ENCRYPTION. Every byte — headers, body, cookies, passwords — travels as plain text.

A 'POST' request hides data from the URL bar (so it is not visible in browser history or server logs), but it is completely visible to ANYONE monitoring the network traffic:

- A hacker on the same Wi-Fi network (man-in-the-middle attack)
- The ISP's network equipment

- Any router between client and server

POST vs GET is about WHERE data appears — URL vs body. It is NOT about encryption.

HTTPS encrypts EVERYTHING: URL, headers, AND body. That is the only secure option.

WHY A IS CRITICALLY WRONG:

POST provides zero encryption. This is one of the most dangerous misconceptions in web development. Never deploy a login form over HTTP. Ever.

WHY C IS WRONG:

GET is LESS appropriate for passwords (data visible in URL, stored in browser history, logged in server access logs). POST is correct — but only over HTTPS.

WHY D IS WRONG:

Routers route packets — they do NOT encrypt them. Only TLS/HTTPS provides encryption.

Part B — Open-Ended Systems Thinking Prompts

Instructions: These are not trick questions. They require students to synthesise knowledge across all four modules and trace real systems end-to-end. Encourage students to draw diagrams and use precise terminology.

SYSTEMS THINKING PROMPT 1 — The 404 Debug Trace

PROMPT:

Trace, step by step, everything that happens when a user types the following URL into their browser and encounters a 404 error. Be specific about every layer of the stack.

URL: <https://www.learningportal.co.ke/courses/machine-learning>

WHAT A STRONG ANSWER COVERS:

1. DNS Resolution: Browser checks cache → OS cache → Recursive Resolver → Root → .ke TLD → Authoritative Server → returns IP. (~50ms)
2. TLS Handshake: Browser verifies SSL certificate. Establishes encrypted tunnel.
3. HTTP Request: GET /courses/machine-learning HTTP/1.1 with headers sent to server.
4. Server Processing: Web server (Apache/Nginx) receives request.
Server looks for a route matching '/courses/machine-learning'.

No course exists in the database / no file at that path.

Server generates a 404 response.

5. HTTP Response: HTTP/1.1 404 Not Found + HTML body (custom error page).

6. Browser Rendering: Browser receives 404 HTML → parses DOM → builds CSSOM → Render Tree → Layout → Paint → user sees the '404 - Page Not Found' page.

BONUS INSIGHT: Why might this cause a 200 response instead of 404 in some systems?

Answer: SPAs (Single Page Applications) often return 200 for all routes and handle routing client-side with JavaScript. The server returns index.html (200 OK) and the JavaScript router shows a 'not found' UI — but the HTTP status was actually 200. This is called a client-side 404 — and it is terrible for SEO.

SYSTEMS THINKING PROMPT 2 — The Performance Audit

PROMPT:

A startup's website loads in 8 seconds on a mobile phone in Kenya on 4G data. Users are leaving before the page loads. As the engineer, trace all the possible bottlenecks at EACH layer of the stack and propose a fix for each.

WHAT A STRONG ANSWER COVERS:

NETWORK LAYER (Module 1):

- Problem: Server is hosted in Europe — high latency (200ms+ round trips).
Fix: Move hosting to an African data centre (AWS Cape Town, Azure Johannesburg) or use a CDN (Cloudflare) with edge nodes in Nairobi.
- Problem: Large page requires many HTTP round trips.
Fix: Use HTTP/2 (multiplexing — multiple files over one connection).

DNS LAYER (Module 2):

- Problem: DNS resolution taking too long (slow authoritative server).
Fix: Use a fast DNS provider (Cloudflare DNS, AWS Route53).
- Problem: Short TTL causing frequent DNS lookups.
Fix: Increase TTL to 3600+ for stable records.

HTTP LAYER (Module 3):

- Problem: HTTPS handshake overhead.
Fix: Enable TLS Session Resumption and HTTP/2 (reduces handshake overhead).

- Problem: Server response time slow (database queries, no caching).
Fix: Add server-side caching (Redis), optimise database queries, add indexes.

BROWSER RENDERING (Module 4):

- Problem: Render-blocking CSS and JavaScript in <head>.
Fix: Add defer to scripts. Inline critical CSS. Lazy-load non-critical CSS.
- Problem: Large uncompressed images.
Fix: Use WebP format, compress images, add lazy loading (loading='lazy').
- Problem: Large JavaScript bundle.
Fix: Code splitting, tree shaking, minification.

SYSTEMS THINKING PROMPT 3 — The Security Incident

PROMPT:

Users of a school's online grading portal report that when they log in over the school's public Wi-Fi, their grades are being changed to 'F' by someone else. The site uses HTTP (not HTTPS). As the security engineer, explain exactly how this attack is happening, what the attacker can see, and what the complete fix looks like.

WHAT A STRONG ANSWER COVERS:

THE ATTACK — Man-in-the-Middle (MitM) on HTTP:

Because the site uses plain HTTP, all traffic is unencrypted.

An attacker connected to the same school Wi-Fi can use a packet-sniffing tool (like Wireshark) to intercept all HTTP traffic on the network.

They can see:

- Username and password in the POST request body (plain text!)
- Session cookies in every subsequent request header
- All grade data in the server's responses

THE GRADE TAMPERING:

The attacker steals a valid session cookie from the network traffic.

They inject this cookie into their own browser (session hijacking).

They are now authenticated as the victim student — no password needed.

They submit a POST request to /api/grades with changed grade data.

THE COMPLETE FIX:

1. Enforce HTTPS immediately — obtain a free certificate from Let's Encrypt.
2. Set Strict-Transport-Security header (HSTS) so browsers always use HTTPS.
3. Set cookies as HttpOnly (JS cannot read them) and Secure (only sent over HTTPS).
4. Implement CSRF tokens on all grade-changing forms.
5. Add server-side authorisation: students can only modify their own data.
6. Enable audit logging: record every grade change with timestamp, IP, and user.

KEY LESSON: The attack was possible at the NETWORK LAYER (Module 1 — unencrypted packets) and the PROTOCOL LAYER (Module 3 — no HTTPS). Security is not one layer — it must be enforced at every layer of the stack simultaneously.

Instructor's Final Word

What you have just taught your students is not a collection of facts. It is a mental model — a way of seeing the invisible machinery that powers the modern world. Every time they open a browser, they now see the packets racing through submarine cables. They see the DNS lookups happening in milliseconds. They see the HTTP conversation in every page load. They see the browser assembling the DOM like a skilled architect.

This is what separates a web developer from a web engineer. Developers know how to use the tools. Engineers understand the system. And engineers build things that last.

THE ENGINEER'S MINDSET — Send Students Into the World With This

When something breaks, the engineer does not panic. They trace the layers:

1. Is it a NETWORK issue? → Check connectivity, ping, traceroute.
2. Is it a DNS issue? → Run nslookup. Check DNS records. Check TTL.
3. Is it an HTTP issue? → Check status codes. Read the response headers.
4. Is it a RENDERING issue? → Open DevTools. Check the Console. Inspect the Network tab.

Every problem has a layer. Find the layer. Fix the problem.

That is the engineering mindset. That is what today was about.

Now go build something extraordinary.

The internet is the greatest engineering achievement in human history.

You are now part of the community that builds it.